ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO TECHNICAL PROGRAMMING

PAPER 2 · PRACTICE B · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 3 Introduction to Technical Programming paper 2 · practice b.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Accept any correct, equivalent code and sensible design choices. Do not penalise small syntax slips (e.g. a missing semicolon) more than once per question unless syntax is being tested. Give follow-through marks.

SECTION A

QUESTION 1

1.1 1.1.1 B ✓ 1.1.2 A ✓ 1.1.3 C ✓ 1.1.4 B ✓ 1.1.5 A ✓ 1.1.6 C ✓ 1.1.7 D ✓ 1.1.8 A ✓ 1.1.9 D ✓ 1.1.10 B ✓ (10)

1.2 1.2.1 True ✓ 1.2.2 False ✓ 1.2.3 True ✓ 1.2.4 True ✓ 1.2.5 False ✓ 1.2.6 True ✓ 1.2.7 False ✓ 1.2.8 True ✓ 1.2.9 False ✓ 1.2.10 True ✓ (10)

1.3 1.3.1 C ✓ 1.3.2 G ✓ 1.3.3 B ✓ 1.3.4 J ✓ 1.3.5 F ✓ 1.3.6 D ✓ 1.3.7 I ✓ 1.3.8 E ✓ 1.3.9 H ✓ 1.3.10 A ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 Inputs ✓✓: start button (pin 2), IR box sensor (pin 3). Outputs ✓✓: conveyor relay / motor (pin 8), green LED (pin 9), yellow LED (pin 10). (4)

2.2 Three states drawn ✓ with arrows: IDLE → RUNNING when the start button is pressed ✓✓; RUNNING → AT_STATION when the IR sensor detects a box ✓; AT_STATION → RUNNING after 2 s ✓; start state IDLE marked / neat labels ✓ (6)

2.3 Marks ✓✓✓✓✓✓✓✓✓✓: state names / constants and a state variable; unsigned long for the start time; pinMode for inputs with INPUT_PULLUP and for the three outputs; switch (or if-else) on the state; IDLE: outputs off, button LOW → RUNNING; RUNNING: relay and green LED on; RUNNING: sensor LOW → record millis(), change to AT_STATION; AT_STATION: relay off, yellow LED on; AT_STATION: millis() − start ≥ 2000 → RUNNING; correct syntax and structure. Example: (10)

```
enum State { IDLE, RUNNING, AT_STATION };
State state = IDLE;
unsigned long stopTime = 0;

void setup() {
  pinMode(2, INPUT_PULLUP);   // start button
  pinMode(3, INPUT);          // IR sensor
  pinMode(8, OUTPUT);         // conveyor relay
  pinMode(9, OUTPUT);         // green LED
  pinMode(10, OUTPUT);        // yellow LED
}

void loop() {
```

```
  switch (state) {
    case IDLE:
      digitalWrite(8, LOW); digitalWrite(9, LOW); digitalWrite(10, LOW);
      if (digitalRead(2) == LOW) state = RUNNING;
      break;
    case RUNNING:
      digitalWrite(8, HIGH); digitalWrite(9, HIGH); digitalWrite(10, LOW);
      if (digitalRead(3) == LOW) { stopTime = millis(); state = AT_STATION; }
      break;
    case AT_STATION:
      digitalWrite(8, LOW); digitalWrite(9, LOW); digitalWrite(10, HIGH);
      if (millis() - stopTime >= 2000) state = RUNNING;
      break;
  }
}
```

[20]

QUESTION 3

3.1 A single reading can be affected by electrical noise or a sudden spike ✓; averaging several readings smooths these out and gives a steadier, more reliable value ✓. (2)

3.2 Function header with int return and pin parameter ✓; array of 10 ✓; loop 10 times ✓; analogRead into the array ✓; add up the values ✓; return total / 10 ✓, e.g. (6)

```
int averageReading(int pin) {
  int values[10];
  long total = 0;
  for (int i = 0; i < 10; i++) {
    values[i] = analogRead(pin);
    total = total + values[i];
  }
  return total / 10;
}
```

3.3 50 ✓: 512 is about half of the input range 0–1 023, so it maps to about half of 0–100 (map uses whole-number maths, so 50,04 becomes 50) ✓ (2)

3.4.1 $\text{tempC} = \text{reading} \times 5.0 \div 1023$ ✓ (voltage) $\times 100$ ✓ ($\div$ 0,01 V per °C), e.g. $\text{float tempC} = \text{reading} * 5.0 / 1023 * 100;$ ✓ (3)

3.4.2 $V = 61 \times 5 \div 1\,023 = 0{,}30$ V ✓; $T = 0{,}298 \div 0{,}01$ ✓ $= 29{,}81$ °C ✓ (3)

[16]

QUESTION 4

4.1 Marks ✓✓✓✓✓✓✓✓✓✓✓: class Motor with private section; int speed; bool forward; public section; constructor with correct name; sets speed = 0 and forward = true; setSpeed with parameter; limits values below 0 to 0; limits values above 255 to 255; getSpeed returns speed; reverse() changes forward with !; display() prints speed and direction. Example: (12)

```
class Motor {
  private:
    int speed;
    bool forward;
  public:
    Motor() { speed = 0; forward = true; }
    void setSpeed(int s) {
      if (s < 0) s = 0;
      if (s > 255) s = 255;
      speed = s;
    }
    int getSpeed() { return speed; }
    void reverse() { forward = !forward; }
    void display() {
      cout << "Speed: " << speed;
      cout << (forward ? " forward" : " reverse") << endl;
    }
};
```

4.2 Two objects created ✓; both speeds set to 200 ✓; rightMotor.reverse() ✓; both displayed ✓, e.g. (4)

```
Motor leftMotor, rightMotor;
leftMotor.setSpeed(200);
rightMotor.setSpeed(200);
rightMotor.reverse();
leftMotor.display();
rightMotor.display();
```

4.3 A class is the blueprint / design that describes the data and functions (Motor describes any motor) ✓; an object is one actual instance made from the class, with its own values (leftMotor and rightMotor) ✓. (2)

[18]

QUESTION 5

5.1 Each error: line and type ✓, correction ✓. Line 2: syntax – missing semicolon: int total = 0; ✓✓. Line 3: logic – semicolon after for() makes the loop empty (and i is then unknown on line 4): for (int i = 1; i <= 5; i++) { ✓✓. Line 6: logic – assignment instead of comparison: if (total == 15) ✓✓. Line 7: syntax – Total is not declared (case sensitive): << total ✓✓. Line 8: syntax – missing closing quotation mark: cout << "Done"; ✓✓ (10)

5.2 Syntax error: breaks the rules of the language and stops the program from compiling ✓. Runtime error: occurs while the program runs and makes it crash, e.g. dividing by zero ✓. Logic error: the program runs but gives the wrong result ✓. (3)

5.3 Any TWO ✓✓: print the values of variables with cout / Serial.println; use a trace table; use breakpoints and step through with a debugger; test with known input and expected output; comment out parts of the code. (2)

5.4 Total is 15 and Done (on the next line) ✓ (1)

[16]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE